

REST Assured

Representational State Transfer is the abbreviation for REST. REST Assured is a Java package used to test and validate Restful Web Services. It supports given, when, and then notations, which are part of behavior-driven development (BDD). It facilitates our integration with testing frameworks like TestNG and Junit.

REST assured is one of the most widely used libraries and is heavily used in most businesses' API test automation. The validation of HTTP Responses received from the server is another feature offered by the Rest assured library. Rest-Assured is a very adaptable library that may be used for testing.

- RESTful web services are tested using the REST-assured Java-based library. To access REST web services, this library acts like a headless client.
- Testing methods employed in dynamic languages like Groovy and Ruby have significantly influenced REST-assured, which was created to make testing and validating REST APIs easier.
- The library offers strong HTTP support, starting with the verbs and basic HTTP operations and going far beyond these fundamentals.

What is REST Assured?

A Java package called REST assured offers a domain-specific language (DSL) for creating, compiling, and dependable testing for RESTful APIs.

REST assured uses highly effective matching techniques, and helping the results you expect to see is also not too difficult. Rest assured provides the methods that are used to obtain data from virtually any area of the request and response, regardless of how complex the underlying JSON structure may be. API automation testing is still very young and specializes in the testing field. However, this does not lessen its significance in the testing process.

Let us check out how the REST assured works –

- Build together a specific HTTP request and send it out.
- Start the process by transmitting the request over the Internet.
- Verify the accuracy of the Response you've got.

REST ASSURED



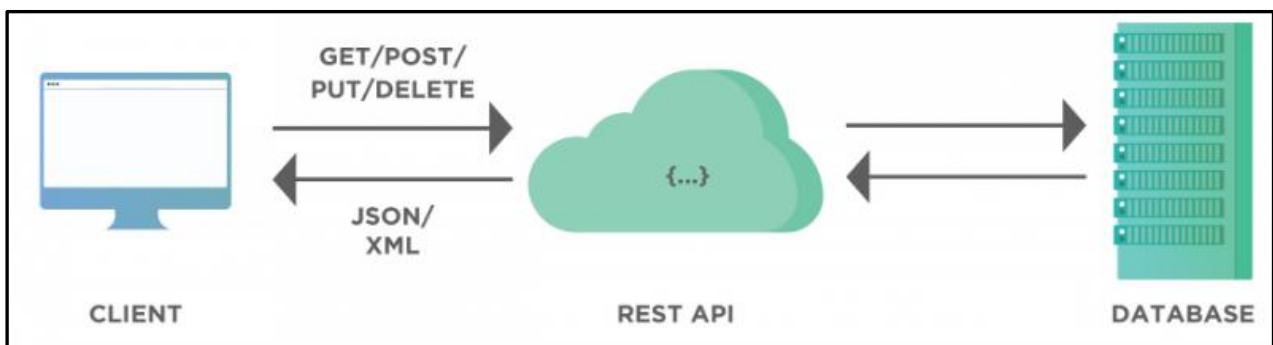
Advantages of Rest Assured:

- It is an Open source Tool.
- Easy parsing and validation of response in JSON and XML.
- It follows BDD keywords like given(), when(), then() which makes code readable and supports clean coding. This feature is available from version 2.0.
- It supports quick assertion for status code and response time.
- It supports assertion to Status Code, Response Time, Headers, cookies, Content-Type, etc.
- It can be easily integrated with other Java libraries like TestNG, JUnit as Test
- It supports JSONPath and XmlPath which helps in parsing JSON and XML response.
- Framework and Extent Report, and Allure Report for reporting purposes.
- It provides quite good support for different authentication mechanisms for APIs.

What is REST API Testing?

REST API testing is a subset of software testing that ensures the correct operation of APIs. An application programming interface (API) allows two programs written in different languages to exchange data and work together. Because it allows us to test the application's business logic before the UI is complete, REST API testing is an integral part of the software development process.

There are wide application programming interface varieties available today. For the time being, we will be using REST Assured to test REST APIs. Testing APIs ensures they are both secure, stable, and functional.



As the above diagram shows, the API works as a medium of communication between a database (that is a part of a server) and a client. When a client sends data through APIs, it goes to the database, do the appropriate operation (such as add, delete, modify etc.) and return the response data that contains response code, header files, cookie info etc.

What are Clients and Resources?

A **Client** can be a software or a person or a system using the APIs to access data from another application server. For example, a developer might access Facebook APIs to embed a live post in their own website. The developer program will call the Facebook API through a browser. So, in this case, the browser acts as a client that is calling the Facebook APIs.

If we visualize this system using the REST diagram above, the client or browser will connect to Facebook-Server over REST API and then get the information required to render it on the screen.

Any smaller unit that can be transformed and addressed through a URL and HTTP method is considered a **Resource**. This resource makes changes to the database. For one application, you may have a lot of resources with all of them assigned a particular task. For example, an online book store may have a resource as a table of the database. A resource in a REST architecture is anything that a client has access to and can modify or update.

Why Do We Need REST Assured?

Information related to the REST Assured Java library leads us to believe in its dependability.

- It is an attractive option for API automation because it is an open-source library with a thriving development community.
- It was challenging to test APIs using dynamic languages like Ruby and Groovy.
- REST service testing and validation are more complex in Java. It gets more straightforward and more effortless with REST Assured. Sending encrypted HTTPS requests with a few lines of Java code is a breeze using this module.
- Automating the backend using rest assured provides reasonable confidence once the fundamentals of API and integration testing have been mastered. As a result, we will have extra time for front-end testing.

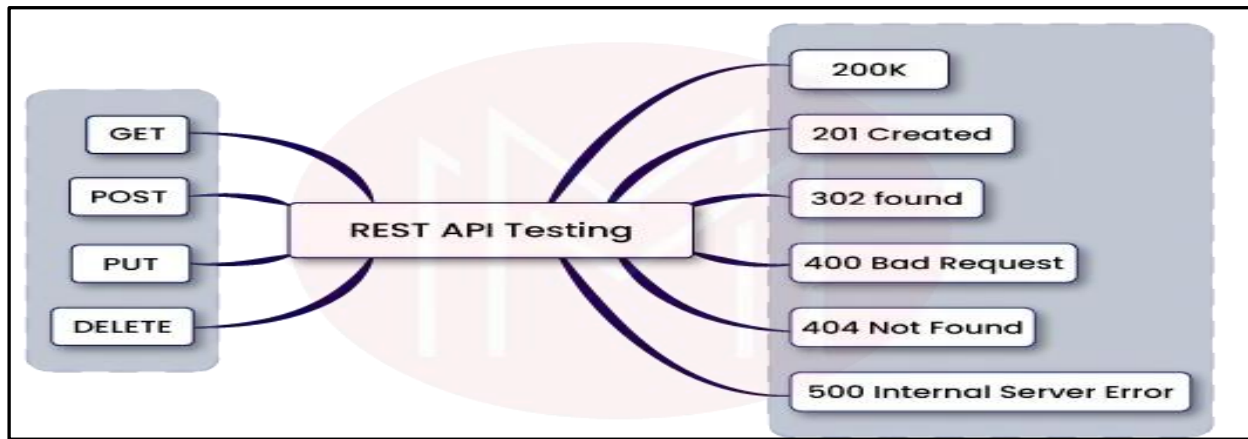
API Testing Process Using REST Assured and POSTMAN Tool:

The ability to safely modernize outdated systems and enterprises toward digital transformation is made possible by APIs. APIs should be fine-tuned to meet the exact needs of a given business scenario. For a successful API, REST API testing is important and helpful. You can manually test an API with the help of POSTMAN, or you can use automation with the help of the REST-assured dependency code already out there.

However, assuming you have a basic knowledge of Java, Rest assured makes it easy to send simple HTTPS requests with user-friendly tweaks. While familiarity with API testing and integration testing is required, Rest Assured offers good backend assurance once those chores have been automated, allowing front-end testing to concentrate on the user interface and client-side procedures. Rest Assured has benefited from the addition of numerous proper methods and libraries as an open-source project, making it a fantastic option for automating APIs.

REST API Interfaces in POSTMAN:

The four most common actions on a RESTful interface are: **create, read, update, and delete (CRUD)**. Tools like POSTMAN and frameworks like Rest Assured and Rest API are used for the manual and automated execution of these tasks, respectively. Additionally, it explains how REST Assured API testing makes it easier to test and validate RestAPI with no complications.



1. Create Operation Performed Using the POST or post() Method:

- Changes to the server can be made via the post() function.
- Here, modifications are implemented on the server by creating new resources there.
- For instance, in the case of Facebook's image-uploading service, we make the necessary adjustments to the server in addition to the already available resources.
- The received Response's status code is 201. "Created" is the reason phrase.

2. Read Operation Performed Using the GET or get() Method:

- The server obtains the data using the get() function.
- This section does not alter the server or the resources already in place.
- Example of a standard search on Google in which we have changed nothing.
- The Response that was received has a Status Code of 200. The Reason-Phrase is "OK."

3. Update Operation Performed Using the PUT or put() Method:

- Put() allows us to make live updates to the server, changing anything we want.
- Here, the server's resources are changed, and consequently, the server is modified.
- We use the preexisting server resources to apply the modifications whenever you edit or update a photo on Facebook.
- The response status code was 200, which indicates that the Response was successfully sent. The Reason-Phrase is "OK."
- The request's path/URL specifies the precise resource that should modify, and a body is also necessary.

4. Delete Operation Performed Using the DELETE or delete() Method:

- We can eliminate anything already on the server by using delete().
- In this section, unused server assets are removed.
- As an illustration, let's say we want to remove a photo of a resource from Facebook.
- The response status code was 204.
- The request path/URL specifies the precise resource to be deleted; a body is unnecessary.

GET – The GET method can be used to extract information from the given server using a given URI. It is worth noting that when you use the GET request, it should only extract the data and not have any other effect on the data. So, no Payload/Body is required.

POST – As the name suggests, a POST request is used to send data to the server. Data like customer information, files, and so on can be sent using HTML forms.

PUT – Using PUT, you will be able to replace all the current representations of the target resource with the uploaded content.

DELETE – It can be used to remove all the current representations of the target resource given by a URI.

Installation and Basic Setup REST Assured Maven Project In Eclipse IDE:

Step1: Download and Install Java:

The Java Development Kit(JDK) allows you to code and run Java programs. The JDK is essentially a toolkit that provides the necessary components to write, compile, and run Java programs. It includes the Java Runtime Environment (JRE), compilers, and various tools such as debuggers and documentation generators.

Rest-Assured needs Java to be installed on the system to run the tests. Refer to the link[<https://www.oracle.com/java/technologies/downloads>] to download JDK. Here, I have a 64-bit system and windows operating system,so I selected – jdk_windows-x64_bin.exe.

Step2: Download and Install Eclipse IDE:

Eclipse IDE is a software that allows you to write your programs and test scripts in multiple programming languages (with Java being the most popular one).

Open the below mentioned link[<https://www.eclipse.org/downloads/packages/>]. Latest version of Eclipse available is Eclipse IDE 2025-09 R Packages.Select – Eclipse IDE for Java Developers and download 64 bit Eclipse and Complete install process.

Step 3: How to install Maven on Windows:

Apache Maven is a software project management and comprehension tool. It uses the concept of a project object model (POM), Maven can manage a project's build and reporting, and documentation from a central piece of information. MAVEN helps us in creating the project structure and managing and downloading the dependencies. We need to define the required dependencies in pom.xml.

Installation Steps:

Step1: To install Apache Maven on the window, we need to download Maven's zip folder from the Official Maven Site[<https://maven.apache.org/download.cgi>]Download apache-maven-3.9.11-bin.zip.

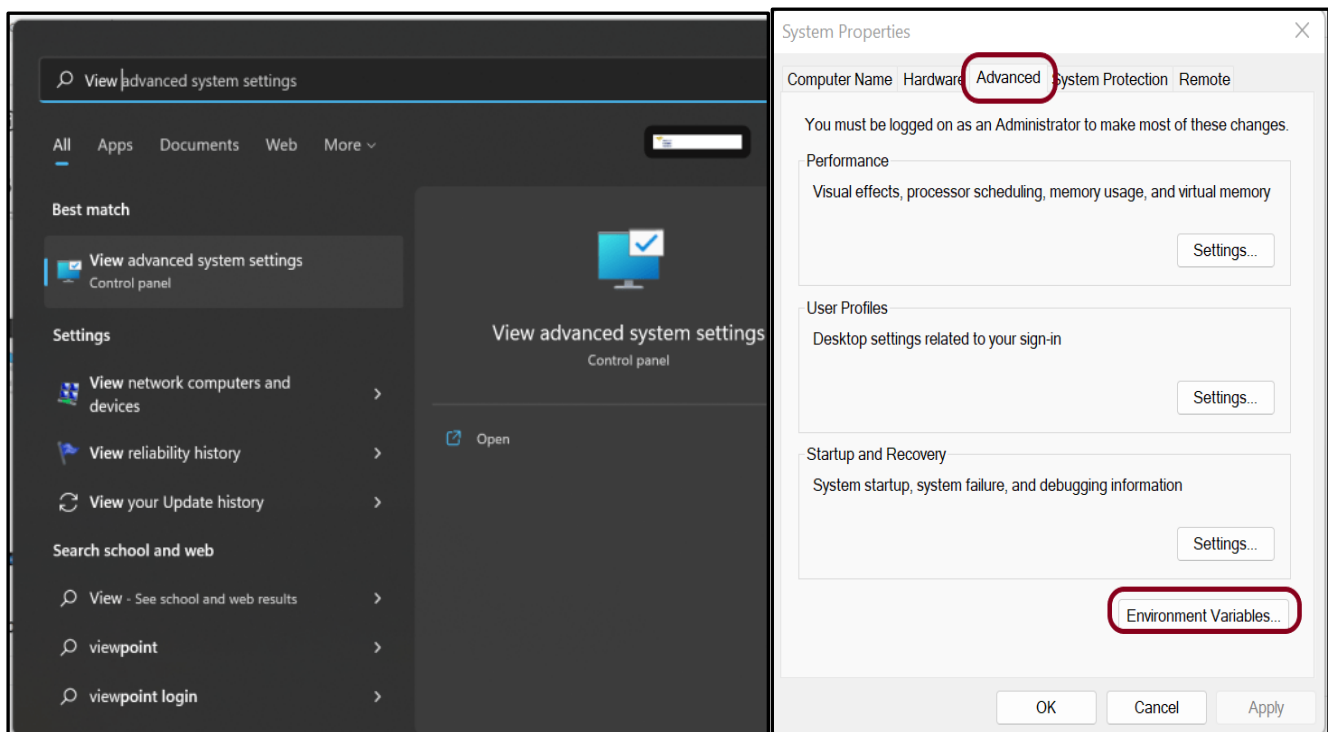
Files			
Maven is distributed in several formats for your convenience. Simply pick a ready-made binary distribution archive and follow the installation instructions . Use a source archive if you intend to build Maven yourself.			
In order to guard against corrupted downloads/installations, it is highly recommended to verify the signature of the release bundles against the public KEYS used by the Apache Maven developers.			
	Link	Checksums	Signature
Binary tar.gz archive	apache-maven-3.9.11-bin.tar.gz	apache-maven-3.9.11-bin.tar.gz.sha512	apache-maven-3.9.11-bin.tar.gz.asc
Binary zip archive	apache-maven-3.9.11-bin.zip	apache-maven-3.9.11-bin.zip.sha512	apache-maven-3.9.11-bin.zip.asc
Source tar.gz archive	apache-maven-3.9.11-src.tar.gz	apache-maven-3.9.11-src.tar.gz.sha512	apache-maven-3.9.11-src.tar.gz.asc
Source zip archive	apache-maven-3.9.11-src.zip	apache-maven-3.9.11-src.zip.sha512	apache-maven-3.9.11-src.zip.asc

Step2: Unzip the downloaded folder and then it will have below-mentioned files. We do not need to install anything, just unzip the folder.

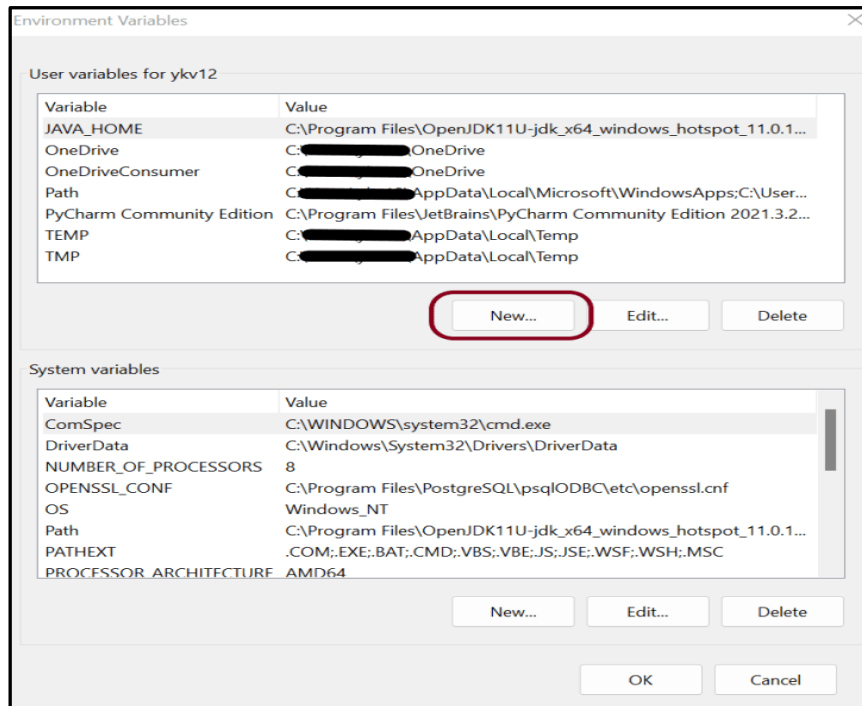
Name	Date modified	Type	Size
▼ Today			
bin	15-11-2025 04:13 PM	File folder	
boot	15-11-2025 04:13 PM	File folder	
conf	15-11-2025 04:13 PM	File folder	
lib	15-11-2025 04:13 PM	File folder	
LICENSE	15-11-2025 04:13 PM	File	9 KB
NOTICE	15-11-2025 04:13 PM	File	5 KB
README.txt	15-11-2025 04:13 PM	Text Document	2 KB

Step3: We need to configure MAVEN_HOME environment variable. Type – “View Adva” in the search option, and we will see the option – View Advanced system setting.

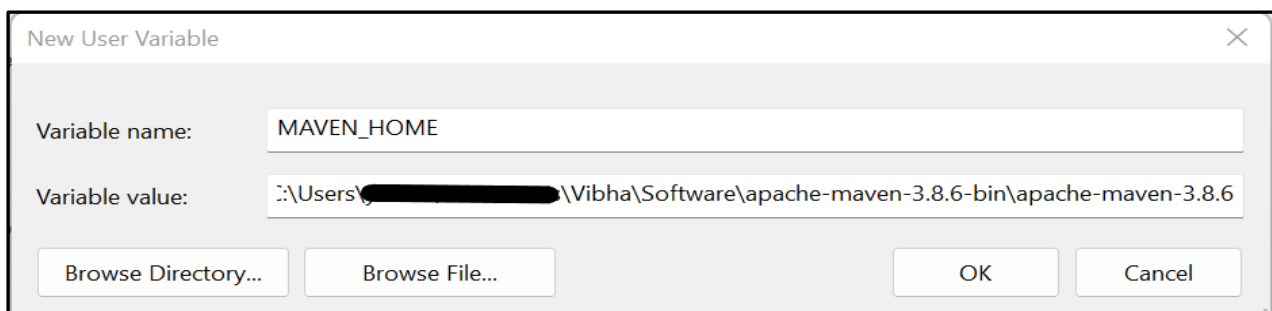
Step4: In the System Properties dialog, select the Advanced tab and click on the Environment Variables button.



Step5: In the “Environment variables” dialog, under Users variables, Click on the New button and add a MAVEN_HOME variable.

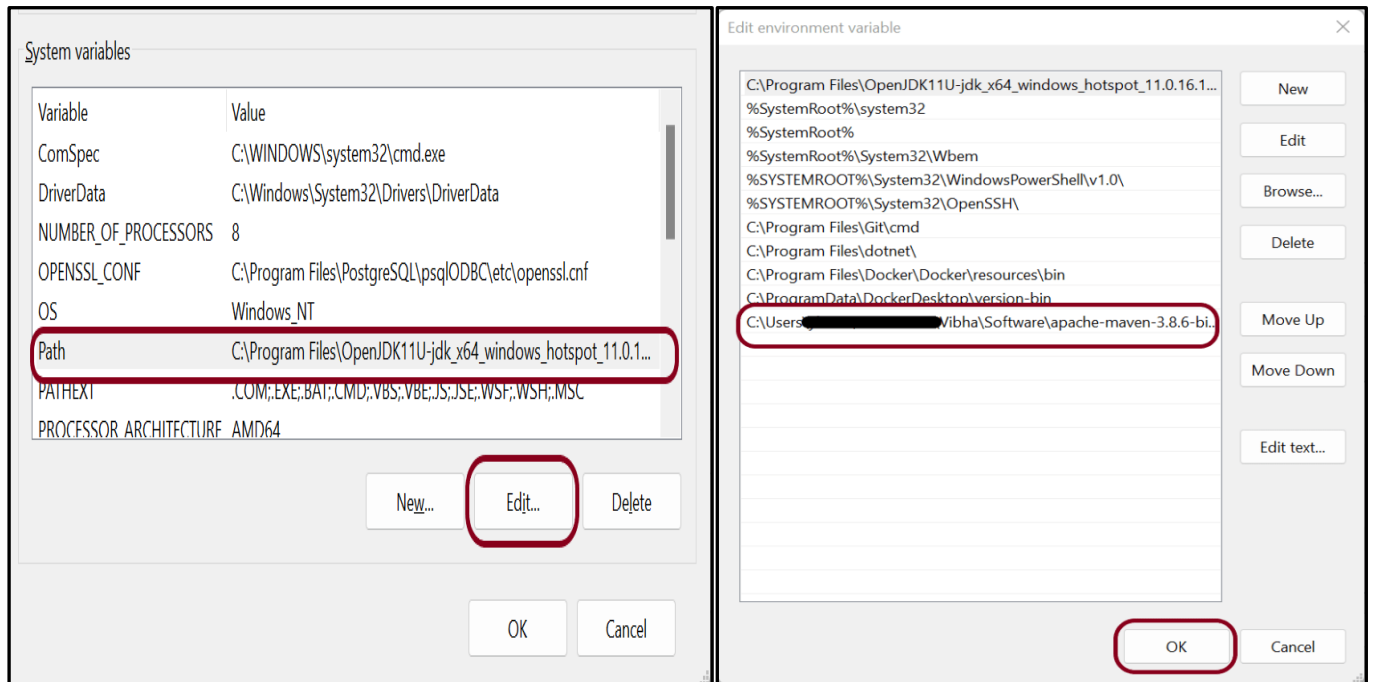


Step6: A dialog box will appear, mentioning Variable Name – MAVEN_HOME and Variable value – mention the path where the Apache folder is placed.

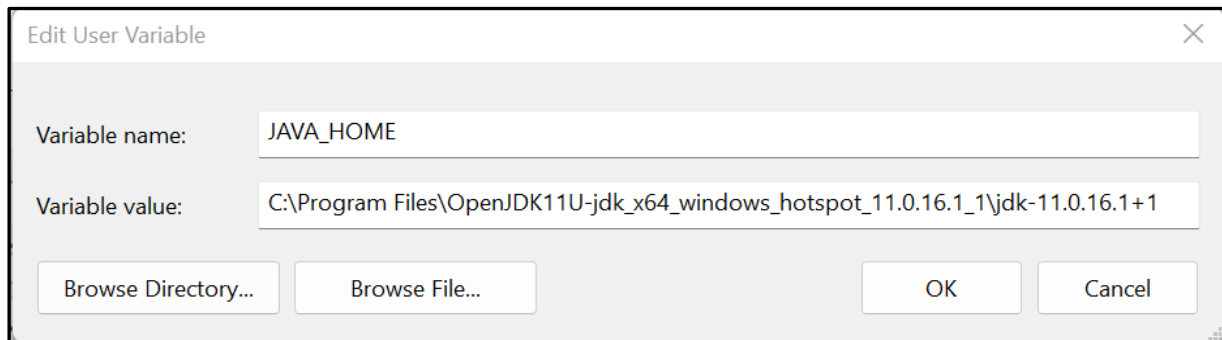


Step7: Add %MAVEN_HOME%\bin (full path till bin where Maven is placed on your machine) to Path present under System variables. Click the New Button present in System Variable and add MAVEN_HOME\bin.

Step8: Once the Path is updated with %MAVEN_HOME%\bin. This is what it will look like.



Step9: We have to make sure that JDK is installed and the JAVA_HOME environment variable is configured. If the JAVA_HOME variable is not configured, then add JAVA_HOME just like MAVEN_HOME in User Variable within Environment Variables.



Step10: Verify if Maven is installed properly on your machine

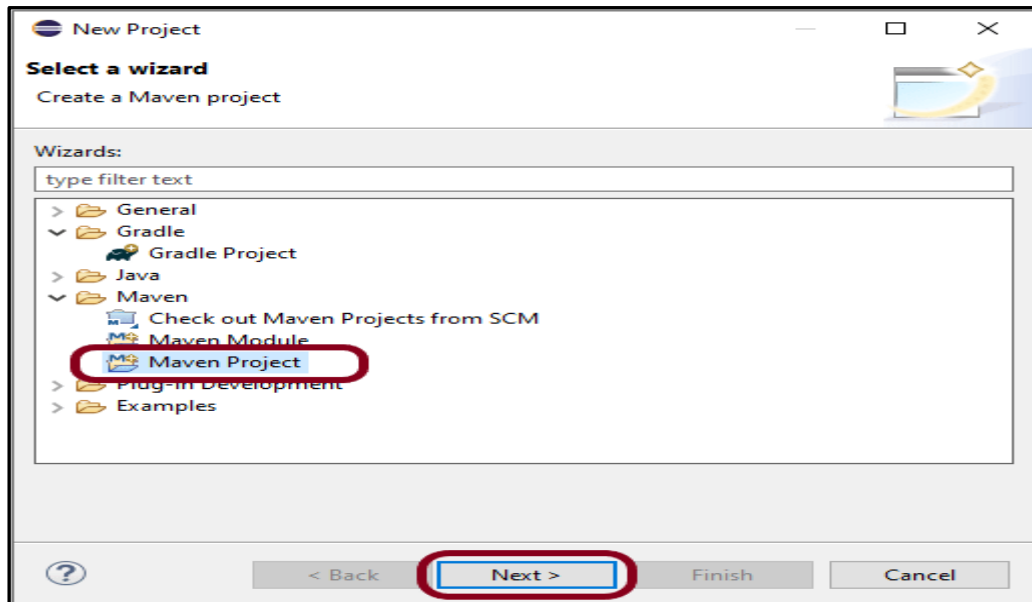
Open the command prompt and type **mvn -version**, then the screen should look something like as shown below screen

```
C:\Users\HP>mvn -version
Apache Maven 3.9.11 (3e54c93a704957b63ee3494413a2b544fd3d825b)
Maven home: C:\Program Files\apache-maven-3.9.11
Java version: 25.0.1, vendor: Oracle Corporation, runtime: C:\Program Files\Java\jdk-25
Default locale: en_IN, platform encoding: UTF-8
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"
```

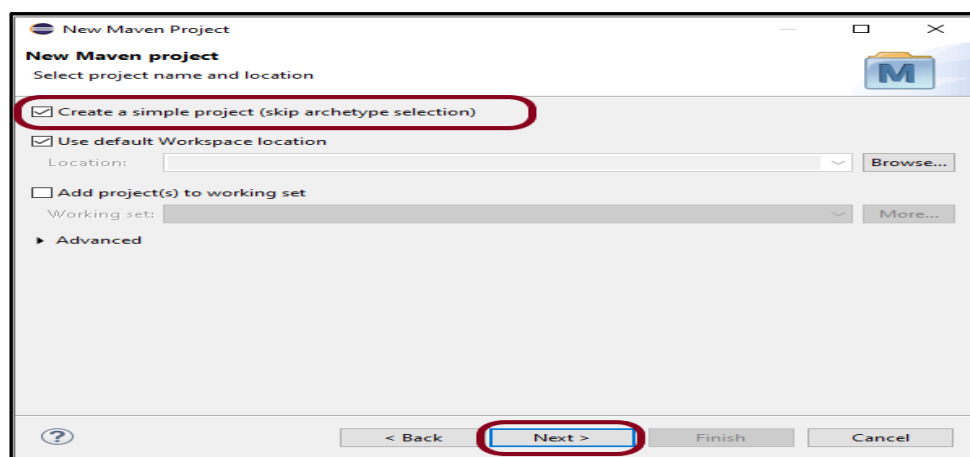
This confirms that Maven is installed successfully and configured on your machine.

Step4: Create a new Maven Project:

Go to File -> New Project-> Maven-> Maven project ->Next



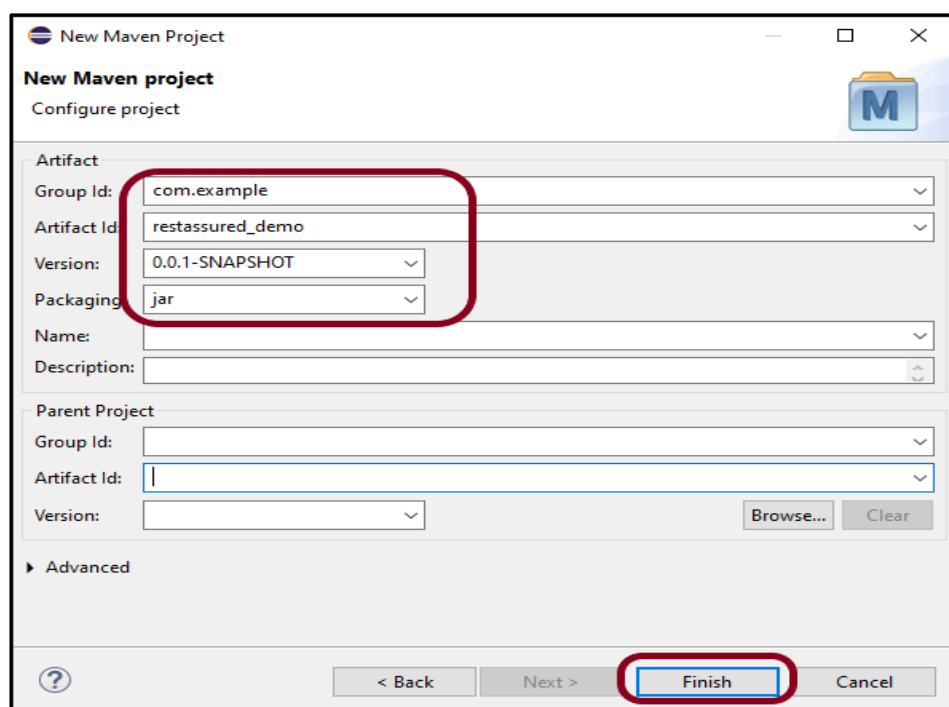
Step4.1: Select “Create a simple project”. Click on the Next Button.



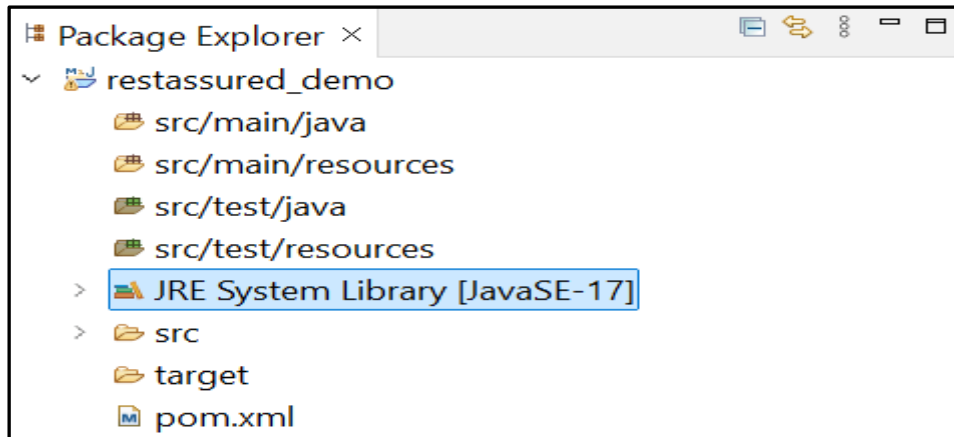
Step4.2: Provide Group Id and Artifact Id and click on the finish button.

Group Id – com.example

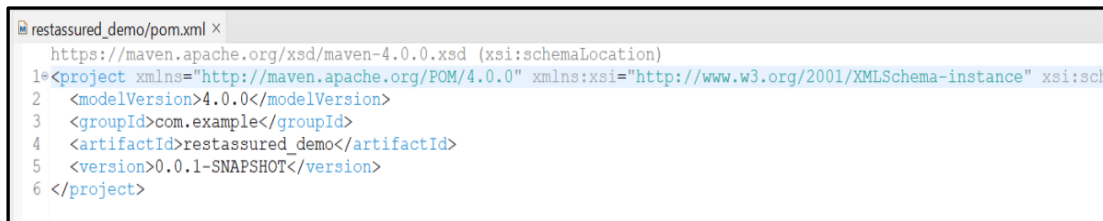
Artifact Id – restassured_demo



Step4.3: Below is the structure of the Maven project in Eclipse.



Step4.4: This is the structure of POM.xml created for the project.



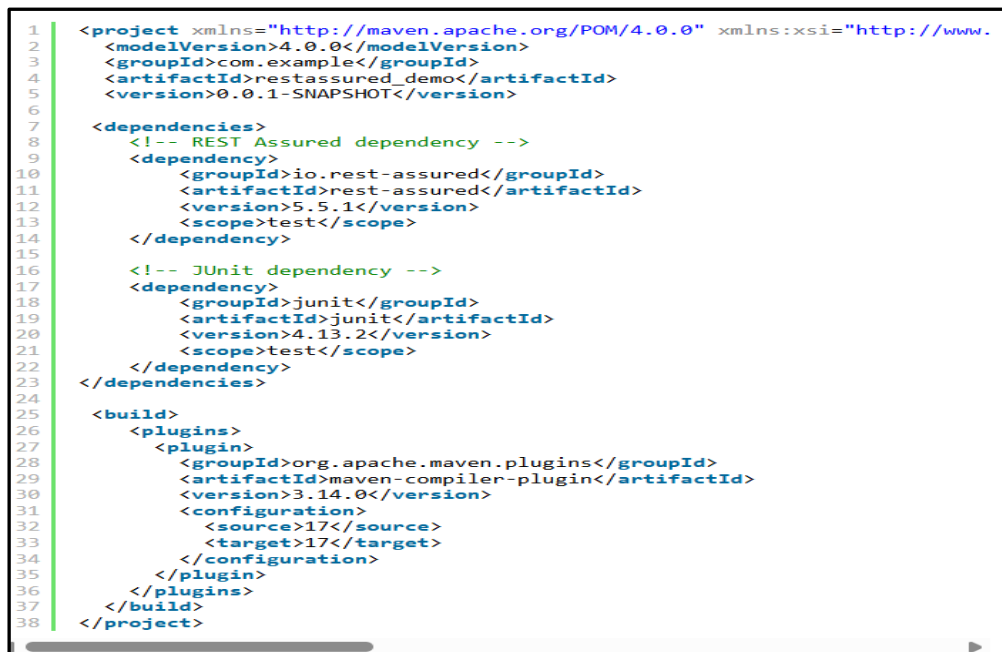
Step5: Add Rest Assured, and JUnit dependencies to the project:

In the Project Explorer, navigate to your Maven project and open pom.xml. Add REST Assured dependency under dependencies in the pom.xml file. The latest version can be found from here [<https://mvnrepository.com/artifact/io.rest-assured>].

Step6: Add Maven Compiler plugin to pom.xml:

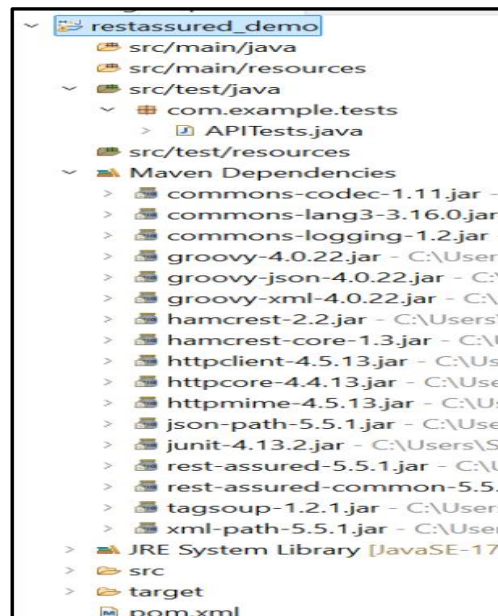
Add Maven Compiler plugin at the build section of the project.

The complete pom.xml looks like as shown below:



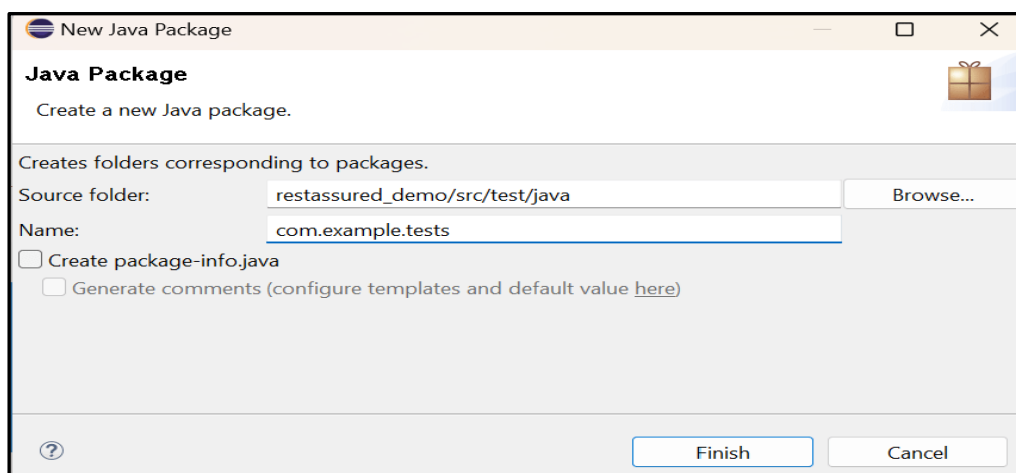
Step7: Update Maven Project:

Right-click on your project in the Eclipse Project Explorer. Choose 'Maven' -> 'Update Project...' and confirm to download the specified dependencies. Below are the Rest Assured, and junit jar files present under Maven Dependencies.



Step8: Create a Test Class:

Right-click on the src/test/java folder, create a new package (e.g., com.example.tests).



Create a new Java class inside this package (e.g., ApiTests.java).

Use the following example to set up a basic REST Assured GET request test.

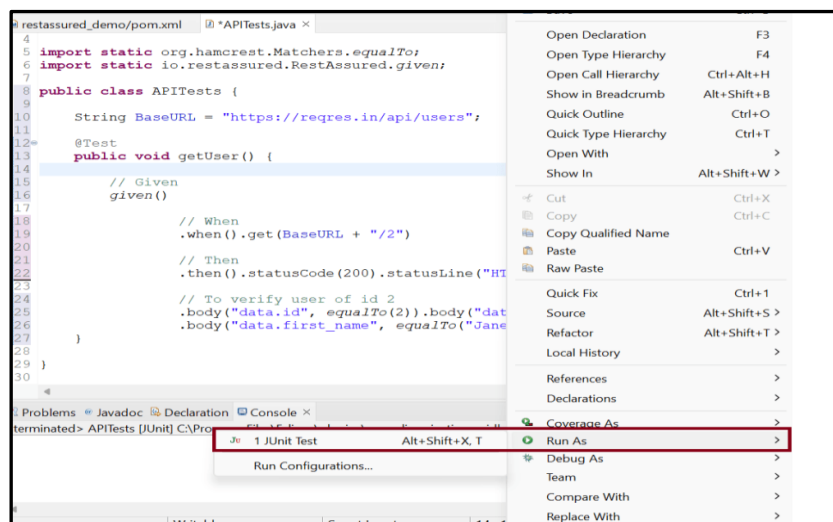
```

1 package com.example.tests;
2
3 import org.junit.Test;
4
5 import static org.hamcrest.Matchers.equalTo;
6 import static io.restassured.RestAssured.given;
7
8 public class APITests {
9
10     String BaseURL = "https://reqres.in/api/users";
11
12     @Test
13     public void getUser() {
14         // Given
15         given()
16
17         // When
18         .when().get(BaseURL + "/2")
19
20         // Then
21         .then().statusCode(200).statusLine("HTTP/1.1 200 OK")
22
23         // To verify user of id 2
24         .body("data.id", equalTo(2)).body("data.email", equalTo("
25         .body("data.first_name", equalTo("Janet")).body("data.las
26
27     }
28
29 }

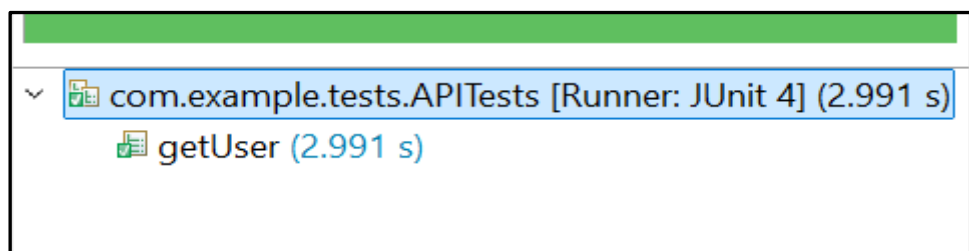
```

Step9: Run the Test:

Right-click on your test class and select Run As -> JUnit Test to execute your REST Assured tests.



The output of the above program is:



How to test GET, POST, PUT, DELETE and PATCH Request using Rest Assured:

RestAssured is a class that consists of many static fields and methods. It supports POST, GET, PUT, DELETE, HEAD, PATCH, and OPTIONS requests and verifies the response to these requests.

1) GET Request using Rest Assured:

Add the below-mentioned dependencies to the pom.xml.

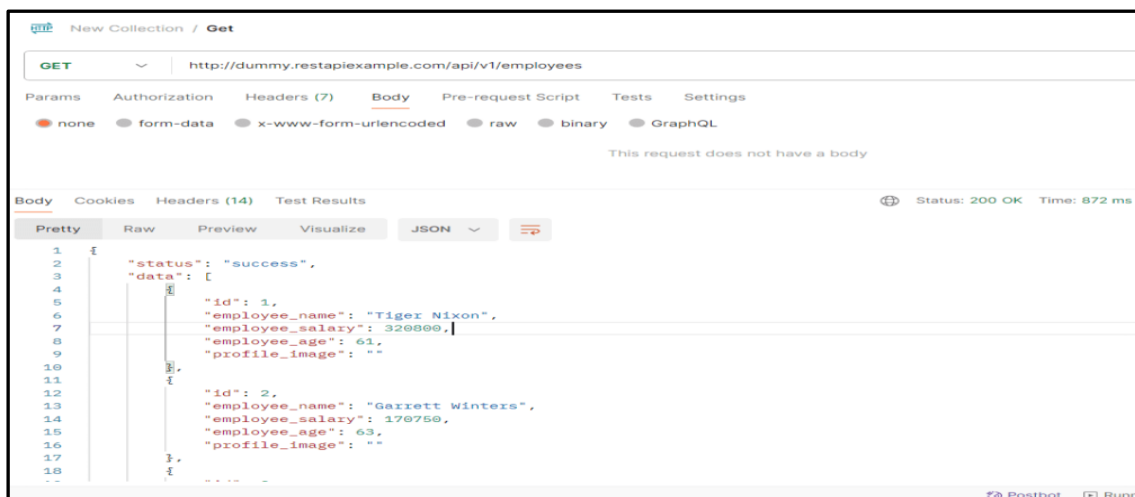
```

<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>io.rest-assured</groupId>
  <artifactId>rest-assured</artifactId>
  <version>5.5.6</version>
  <scope>test</scope>
</dependency>

```

Below is an example of GET Request in Postman.

[<https://dummy.restapiexample.com/api/v1/employees>]



Below are the steps to test a GET Request using Rest Assured:

Step1: Specify the base URL to the RESTful web service using the RestAssured class.

```
RestAssured.baseURI = "http://dummy.restapiexample.com/api/v1/employees";
```

Step2: Every Request in the Rest-Assured library is represented by an interface called RequestSpecification. This interface allows modification of the request, like adding headers or adding authentication details.

```

requestSpecification = RestAssured.given();
RequestSpecification is imported from the package:
import io.restassured.specification.RequestSpecification;

```

Step3: Send the request to the server and receive the response to the request made by REST Assured. This response contains every detail returned by hitting request i.e. response body, response headers, status code, status lines, cookies, etc.

```
response = requestSpecification.get();  
The response is imported from package:  
import io.restassured.response.Response;
```

Step4: To validate a response like status code or value, we need to acquire a reference. This reference should be of type ValidatableResponse. ValidatableResponse is an interface. A validatable response to a request made by, REST Assured. ValidatableResponse is imported from the package:

```
import io.restassured.response.ValidatableResponse;
```

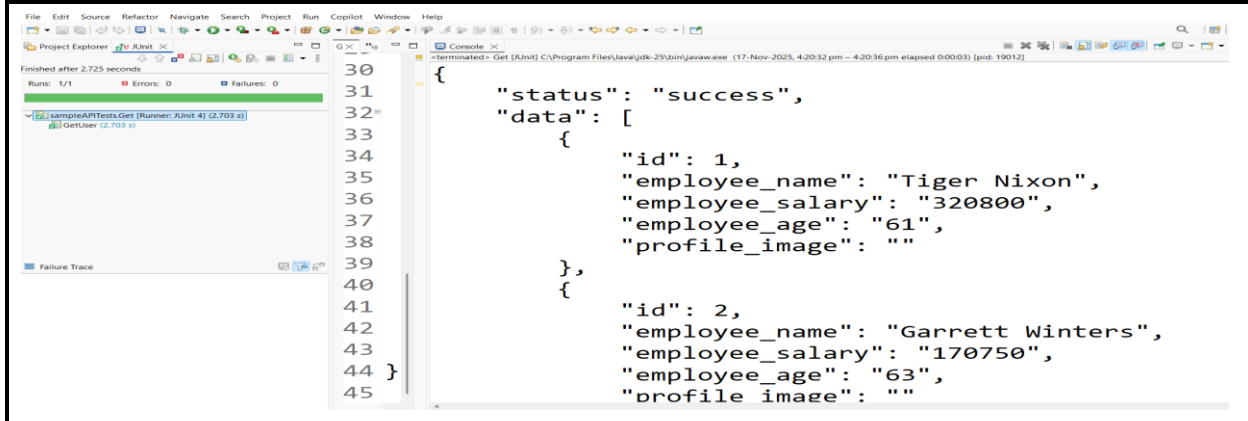
PrettyPrint(): It prints the response body if possible and returns it as a string. Pretty printing is possible for content-types JSON, XML, and HTML.

Below is an example of creating a test in Non-BDD format. I have used ValidatableResponse for the assertion of the status. It is also used for the status line of the Response.

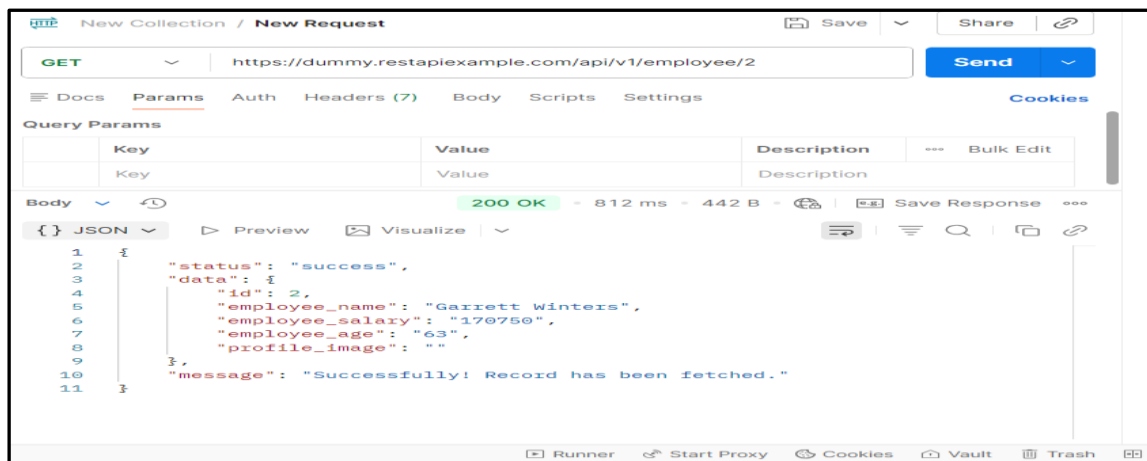
```
import io.restassured.RestAssured;  
import io.restassured.response.Response;  
import io.restassured.response.ValidatableResponse;  
import io.restassured.specification.RequestSpecification;  
import org.junit.Assert;  
import org.junit.Test;  
public class Get_NonBDDResponseDemo {  
    RequestSpecification requestSpecification;  
    Response response;  
    @Test  
    public void verifyStatusCode() {  
        RestAssured.baseURI = "http://dummy.restapiexample.com/api/v1/employees";  
        // Create a request specification  
        requestSpecification = RestAssured.given();  
        // Calling GET method  
        response = requestSpecification.get();  
        // Let's print response body.  
        String resString = response.prettyPrint();  
        System.out.println("Response Details : " + resString);  
    }  
}
```

```
// Get status line
String statusLine = response.getStatusLine();
Assert.assertEquals(statusLine, "HTTP/1.1 200 OK");

// Get status code
int statusCode = response.getStatusCode();
Assert.assertEquals(statusCode, 200);
} }
```



Below is an example of a specific user in GET Request in Postman.



Below is the test implemented in BDD Format. In this test, I am asserting the data of Employee of Id 2. I have validated the name of the employee as well as the response message.

1. equalTo is used for assertion, and is imported from a static hamcrest package:

```
import static org.hamcrest.Matchers.equalTo;
```

2. given is a static import from package:

```
import static io.restassured.RestAssured.given;
```

Let us see this with an example:

```
package EmployeeData;

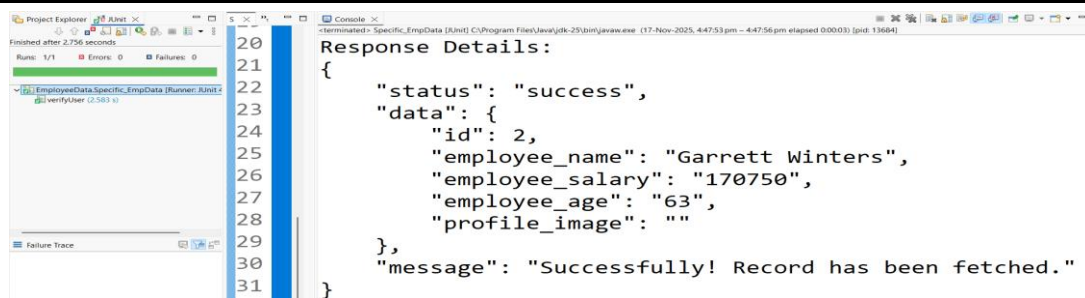
import io.restassured.response.Response;
```

```

import static io.restassured.RestAssured.given;
import static org.hamcrest.Matchers.equalTo;
import org.junit.Test;

public class Specific_EmpData {
    @Test
    public void verifyUser() {
        // Given
        Response response =
        given()
        // When
        .when().get("http://dummy.restapiexample.com/api/v1/employee/2")
        // Then
        .then().statusCode(200).statusLine("HTTP/1.1 200 OK")
        // To verify booking id at index 3
        .body("data.employee_name", equalTo("Garrett Winters"))
        .body("message", equalTo("Successfully! Record has been fetched.")).
        extract()
        .response();
        // PRETTY PRINT RESPONSE BODY
        String resString = response.asPrettyString();
        System.out.println("Response Details:\n" + resString);
    }
}

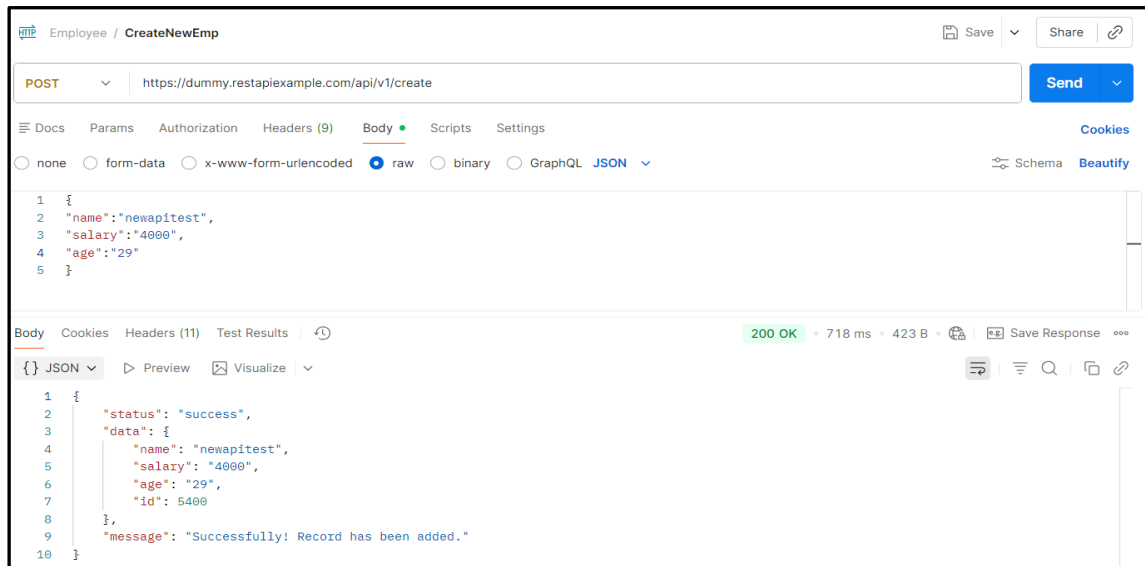
```



2) POST Request using Rest Assured:

An HTTP POST method is used to create a new resource in the collection of resources. The request body is passed as a JSON/XML or in a different format. If a resource is created successfully by the endpoint or server, it returns a status code 201(Created). It also provides a Location header with a link to the newly-created resource with the 201 HTTP status.

Below is an example of a Request performing POST operation in Postman:



Test implemented in BDD Format

1. equalTo is used for assertion, and is imported from a static hamcrest package:

import static org.hamcrest.Matchers.equalTo;

2. given is a static import from package:

import static io.restassured.RestAssured.given;

Below is an example of a BDD Test.

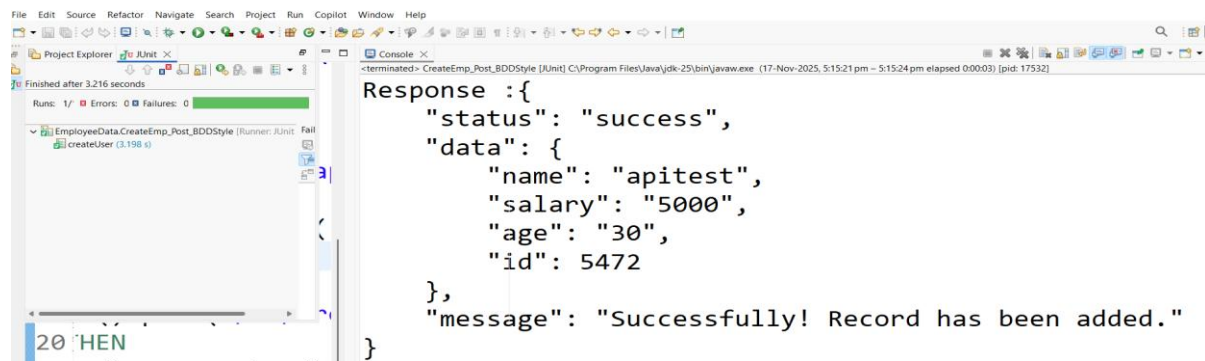
```
import io.restassured.http.ContentType;
import io.restassured.response.ValidatableResponse;
import org.junit.Test;
import static io.restassured.RestAssured.given;
import static org.hamcrest.CoreMatchers.equalTo;

public class POST_BDDDemo {
    ValidatableResponse validatableResponse;

    @Test
    public void createUser() {
        String json = "{\"name\":\"apitest\",\"salary\":\"5000\",\"age\":\"30\"}";
        // GIVEN
        validatableResponse = given()
            .baseUrl("https://dummy.restapiexample.com/api")
            .contentType(ContentType.JSON)
            .body(json)
        // WHEN
        .when()
        .post("/v1/create")
    }
}
```

```
// THEN
.then()
.assertThat().statusCode(200).body("data.name", equalTo("apitest"))
.body("message", equalTo("Successfully! Record has been added.));
System.out.println("Response :" + validatableResponse.extract().asPrettyString());
} }
```

The below image shows the test result of the above test.



Explanation:

1. This string represents the JSON payload that will be sent in the body of the POST request. It includes fields like name, salary, and age.

String json = "{\"name\":\"apitest\",\"salary\":\"5000\",\"age\":\"30\"}\";

2. GIVEN: Specifies the initial conditions or setup for the test.

“given()”: Starts the BDD chain.

Sets the base URI for the API endpoint.

.baseUrl("https://dummy.restapiexample.com/api")

Specifies that the content type of the request is JSON.

.contentType(ContentType.JSON)

Includes the JSON payload in the request body.

.body(json)

3. WHEN: Describes the action or event that triggers the response.

.when().post("/v1/create"): Sends a POST request to the specified endpoint `/v1/create` relative to the base URI.

4. THEN: Asserts expected outcomes of the request.

“.then()”: Starts the section for response validation.

Asserts that the HTTP status code of the response is 200, which typically indicates success.

".assertThat().statusCode(200)"

Asserts that the “name” field in the “data” object of the response matches “apitest”.
Asserts that the “message” field in the response is as expected.

```
.body("data.name", equalTo("apitest"))
```

```
.body("message", equalTo("Successfully! Record has been added."))
```

Test implemented Normal Format [Non-BDD Format]: Below are the steps to test a POST Request using Rest Assured:

Step1: Specify the base URL to the RESTful web service using the RestAssured class.

```
RestAssured.baseURI = "https://dummy.restapiexample.com/api/v1/create";
```

Step2: Every Request in the Rest-Assured library is represented by an interface called RequestSpecification. This interface allows modifying the request, like adding headers or adding authentication details. Use the RestAssured class to generate a RequestSpecification.

```
requestSpecification = RestAssured.given();  
RequestSpecification is imported from package:  
import io.restassured.specification.RequestSpecification;
```

Step3: Set the content type. This step specifies the format in which the request payload will be sent to the server. Here, the Content-Type is JSON.

```
requestSpecification.contentType(ContentType.JSON);  
contentType is imported from restassured.http package:  
import io.restassured.http.ContentType;
```

Step4: Pass Request Body as String.

```
requestSpecification.body(jsonString);
```

Step5: Send the POST request to the server. Then receive the response of the request made by REST Assured. This response contains every detail returned by hitting request i.e. response body, response headers, status code, status lines, cookies, etc. The response is imported from package:

```
import io.restassured.response.Response;
```

Step6: To validate a response like status code or value, we need to get the reference of type ValidatableResponse.

ValidatableResponse is an interface. A validatable response to a request made by, REST Assured. ValidatableResponse is imported from package:

```
import io.restassured.response.ValidatableResponse;
```

PrettyPrint() – It prints the response body if possible and returns it as a string. Pretty printing is possible for content-types JSON, XML, and HTML.

Below is the example of testing a POST request in Non-BDD format. I have used ValidatableResponse for the assertion of status. It is also used for the status line and body of the Response.

```
import io.restassured.RestAssured;
import io.restassured.http.ContentType;
import io.restassured.response.Response;
import io.restassured.response.ValidatableResponse;
import io.restassured.specification.RequestSpecification;
import org.junit.Test;
import static org.hamcrest.CoreMatchers.equalTo;

public class POST_NonBDDDemo {
    RequestSpecification requestSpecification;
    Response response;
    ValidatableResponse validatableResponse;

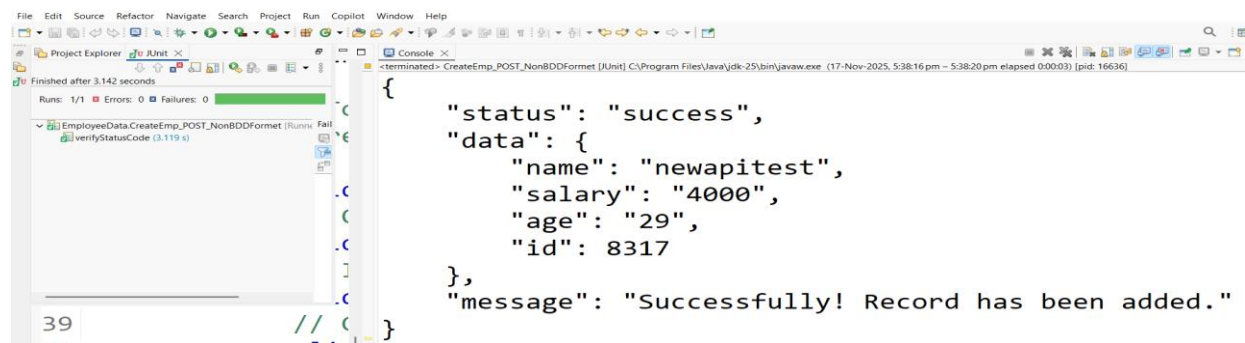
    @Test
    public void verifyStatusCode() {
        String jsonString = "{\"name\":\"newapitest\",\"salary\":\"4000\",\"age\":\"29\"}";
        RestAssured.baseURI = "https://dummy.restapiexample.com/api/v1/create";
        // Create a request specification
        requestSpecification = RestAssured.given();
        // Setting content type to specify format in which request payload will be sent.
        requestSpecification.contentType(ContentType.JSON);
        // Adding body as string
        requestSpecification.body(jsonString);
        // Calling POST method
        response = requestSpecification.post();
        // Let's print response body.
        String responseString = response.prettyPrint();
        /*To perform validation on response, we need to get ValidatableResponse type of response */
        validatableResponse = response.then();
        // Check status code
        validatableResponse.statusCode(200);
        // It will check if status line is as expected
    }
}
```

```

validatableResponse.statusLine("HTTP/1.1 200 OK");
// Check response body - name attribute
validatableResponse.body("data.name", equalTo("newapitest"));
// Check response body - message attribute
validatableResponse.body("message", equalTo("Successfully! Record has been
added."));
}

```

The below image shows the test result of the above test.

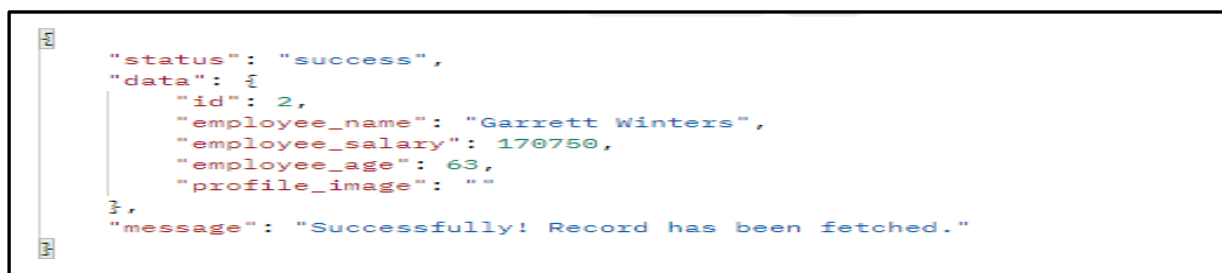


3) PUT Request using Rest Assured:

The HTTP PUT API is primarily used to update existing resources. If the resource does not exist, then API may decide to create a new resource or not (Depending on API development). If a new resource has been created by the PUT API, the origin server MUST inform the user agent via the HTTP response code 201 (Created) response, and if an existing resource is modified, either the 200 (OK) or 204 (No Content) response codes SHOULD be sent to indicate successful completion of the request.

Below is an example of a Request performing PUT operation in Postman:

Below is the response received for Employee with id 2.



I want to change the employee_salary to 99999. Below is the example for the test to update employee_salary.

POSTMAN STEPS:

→ Identify the API Endpoint: Use the Dummy API update endpoint:
<https://dummy.restapiexample.com/api/v1/update/2>
 Here 2 = employee ID
 This endpoint accepts PUT or PATCH for updating details

→ Select HTTP Method

Choose:

✓ PUT → Full record update

or

✓ PATCH → Partial update

Here we use PUT

→ Add Required Headers

Key: Content-Type Value: application/json

→ Prepare the JSON Body

To update salary → 99999:

```
{
  "employee_name": "Garrett Winters",
  "employee_salary": 99999,
  "employee_age": 63,
  "profile_image": ""
}
```

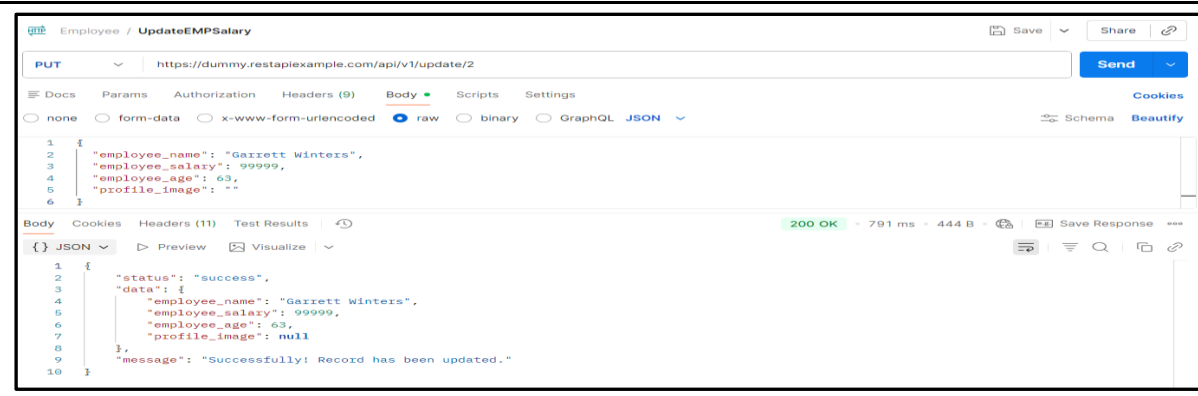
→ Send the Request

Paste URL->Choose PUT->Add header->Add JSON body->Click Send

→ Expected Successful Response

```
{
  "status": "success",
  "data": {
    "id": "2",
    "employee_name": "Garrett Winters",
    "employee_salary": 99999,
    "employee_age": 63,
    "profile_image": ""
  },
  "message": "Successfully! Record has been updated."
}
```

✓ Status Code: 200 OK



Below are the steps to test a PUT Request using Rest Assured:

Now, let us convert the same test into **BDD** format. In the below example, in the first part, we have retrieved the details of the employee with ID 2, and in the second part, we have updated the value of employee_salary to 99999.

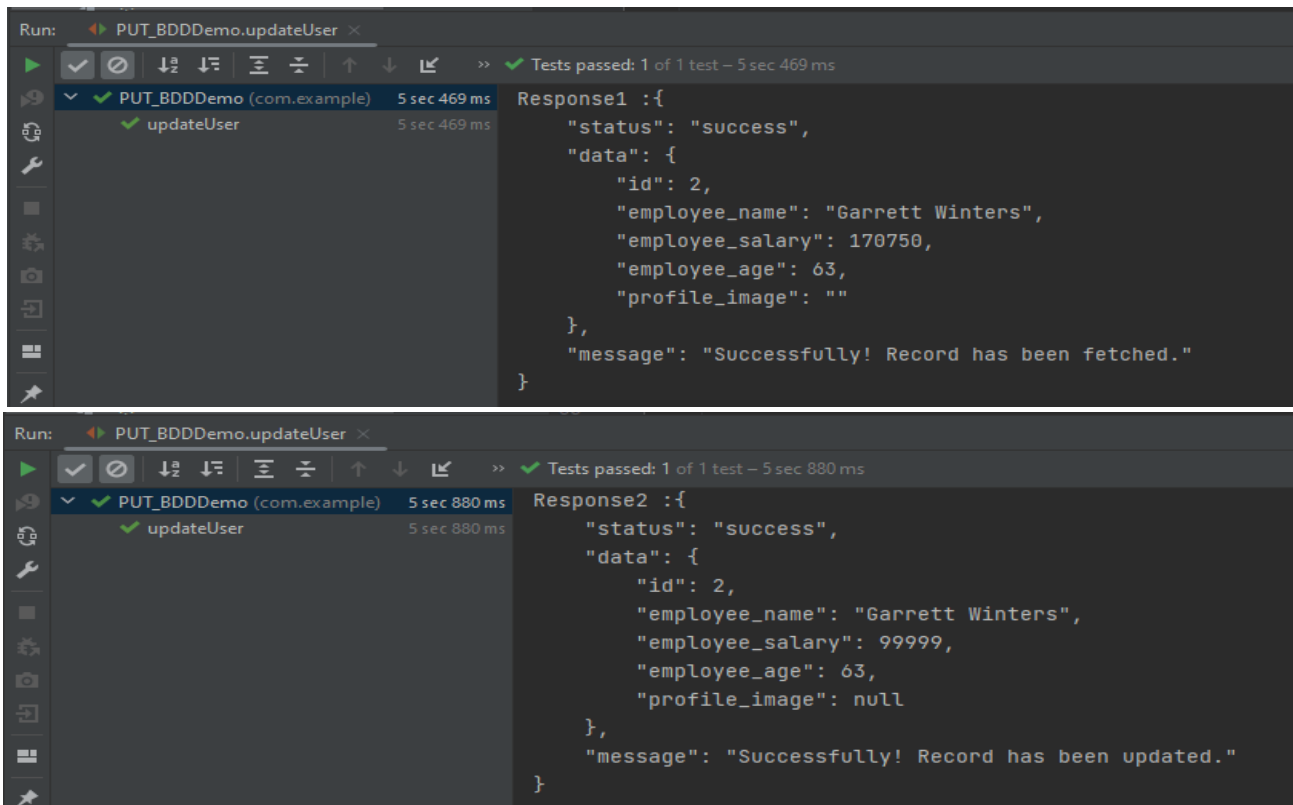
```
package EmployeeData;
import io.restassured.http.ContentType;
import io.restassured.response.Response;
import io.restassured.response.ValidatableResponse;
import io.restassured.specification.RequestSpecification;
import org.junit.Test;
import static io.restassured.RestAssured.given;
import static org.hamcrest.CoreMatchers.equalTo;
public class PUT_Update_EmpSalary_BDDStyle
{
    RequestSpecification requestSpecification;
    Response response;
    ValidatableResponse validatableResponse, validatableResponse1;
    @Test
    public void updateUser() {
        // To get the detail of employee with id 2
        validatableResponse = given()
        .baseUrl("https://dummy.restapiexample.com/api/v1/employee/2")
        .contentType(ContentType.JSON)
        .when()
        .get()
        .then()
        .assertThat().statusCode(200);
        System.out.println("Response1 :" + validatableResponse.extract().asPrettyString());
        String jsonString = "{\"id\": 2,\r\n"
        + "    \"employee_name\": \"Garrett Winters\", \r\n"
        + "    \"employee_salary\": 99999, \r\n"
        + "    \"employee_age\": 63, \r\n"
        + "    \"profile_image\": \"\"}";
        // Update employee_salary
```

```

validatableResponse1 = given()
    .baseUrl("https://dummy.restapiexample.com/api/v1/update/2")
    .contentType(ContentType.JSON)
    .body(jsonString)
    .when()
    .put()
    .then()
    .assertThat().statusCode(200)
    .body("data.employee_salary", equalTo(99999))
    .body("message", equalTo("Successfully! Record has been updated.));
System.out.println("Response2 :"+ validatableResponse1.extract().asPrettyString());
}
}

```

The output of the above Script is:



Below is the example of testing a PUT request in Non-BDD format.

```

import io.restassured.RestAssured;
import io.restassured.http.ContentType;
import io.restassured.response.Response;
import io.restassured.response.ValidatableResponse;
import io.restassured.specification.RequestSpecification;

```

```

import org.junit.Test;
import static org.hamcrest.CoreMatchers.equalTo;

public class PUT_NonBDDDemo {
    RequestSpecification requestSpecification;
    Response response;
    ValidatableResponse validatableResponse;

    @Test
    public void updateUser() {
        String jsonString = "{ \"id\": 2, \r\n"
+ "    \"employee_name\": \"Garrett Winters\", \r\n"
+ "    \"employee_salary\": 99999, \r\n"
+ "    \"employee_age\": 63, \r\n"
+ "    \"profile_image\": \"\" }";

        RestAssured.baseURI = "https://dummy.restapiexample.com/api/v1/update/2";
        // Create a request specification
        requestSpecification = RestAssured.given();
        // Setting content type to specify format in which request payload will be sent.
        requestSpecification.contentType(ContentType.JSON);
        // Adding body as string
        requestSpecification.body(jsonString);
        // Calling PUT method
        response = requestSpecification.put();
        // Let's print response body.
        String responseString = response.prettyPrint();
        /*
        * To perform validation on response, we need to get ValidatableResponse type of
        * response
        */
        validatableResponse = response.then();
        // Get status code
        validatableResponse.statusCode(200);
        // It will check if status line is as expected
        validatableResponse.statusLine("HTTP/1.1 200 OK");
        // Check response - name attribute
    }
}

```

```

validatableResponse.body("data.employee_salary", equalTo(99999));
// Check response - message attribute
validatableResponse.body("message", equalTo("Successfully! Record has been
updated."));
} }

```

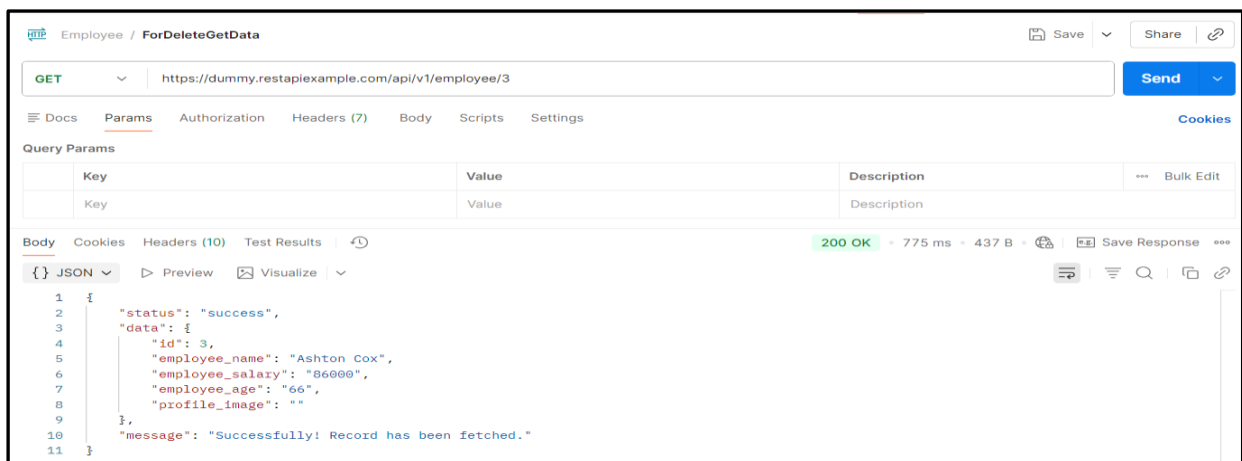
The output of the above Script is:



4) DELETE in Rest Assured:

An HTTP DELETE method is used to delete an existing resource from the collection of resources. The DELETE method requests the origin server to delete the resource identified by the Request-URI. On successful deletion of a resource, it returns 200 (OK) and 204 (No Content) status codes. It may return as 202 (Accepted) status code if the request is queued.

Let's see the existing details of an **Employee ID 3** using Postman:



Below is an example of a Request performing DELETE operation for id-3 in Postman:

API Endpoint: To delete employee with ID 3, use:

<https://dummy.restapiexample.com/api/v1/delete/3>

✓ DELETE request must always point to /delete/{id}

HTTP Method: Select: DELETE

Headers: Usually DELETE does not require headers.

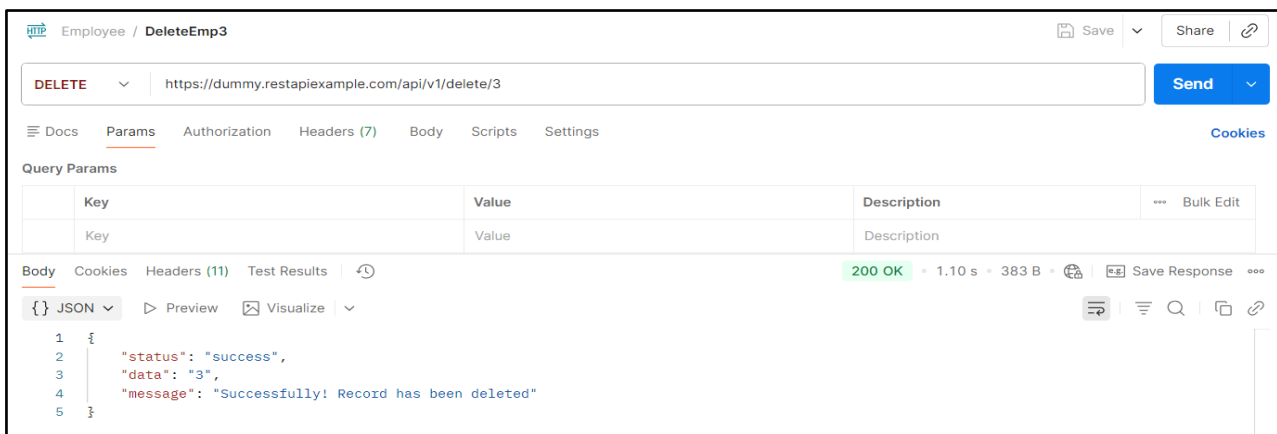
Body for DELETE: DELETE requests do not require a body in this API.

→ Leave the Body section empty.

Expected Response (Success): If employee ID 3 is deleted successfully, response looks like:

```
{
  "status": "success",
  "data": "3",
  "message": "Successfully! Record has been deleted"
}
```

✓ Status Code: 200 OK



Let's write DELETE request in REST Assured in BDD Format:

The steps to test the DELETE request are similar to any API request like GET, POST, or PUT.

```
package EmployeeData;
import io.restassured.http.ContentType;
import io.restassured.response.ValidatableResponse;
import org.junit.Test;
import static io.restassured.RestAssured.given;
import static org.hamcrest.CoreMatchers.equalTo;

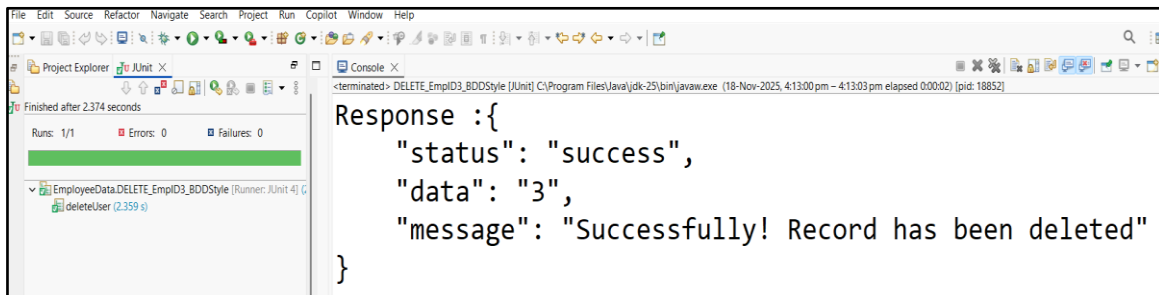
public class DELETE_EmpID3_BDDStyle {
    ValidatableResponse validatableResponse;

    @Test
    public void deleteUser() {
        validatableResponse =
        given().baseUrl("https://dummy.restapiexample.com/api/v1/delete/3")
```

```

.contentType(MediaType.JSON).when().delete().then().assertThat().statusCode(200)
.body("message", equalTo("Successfully! Record has been deleted"));
System.out.println("Response :" + validatableResponse.extract().asPrettyString());
} }

```



Explanation:

1. **GIVEN:** Specifies the initial conditions or setup for the test.

“given()”: Starts the BDD chain.

Sets the base URI for the API endpoint.

```
.baseUrl("https://dummy.restapiexample.com/api/v1/delete/3")
```

Specifies that the content type of the request is JSON.

```
.contentType(MediaType.JSON)
```

2. **WHEN:** Describes the action or event that triggers the response.

.when().delete() – Executes the DELETE HTTP request against the configured endpoint URL.

3. **THEN:** Asserts expected outcomes of the request.

“.then()”: Starts the section for response validation.

Asserts that the HTTP status code of the response is 200, which typically indicates success.

```
assertThat().statusCode(200)
```

Asserts that the “message” field in the response matches “Successfully! Record has been deleted”.

```
body("message", equalTo("Successfully! Record has been deleted"));
```

Let’s write DELETE request in REST Assured in Non BDD Format for id 3:

```

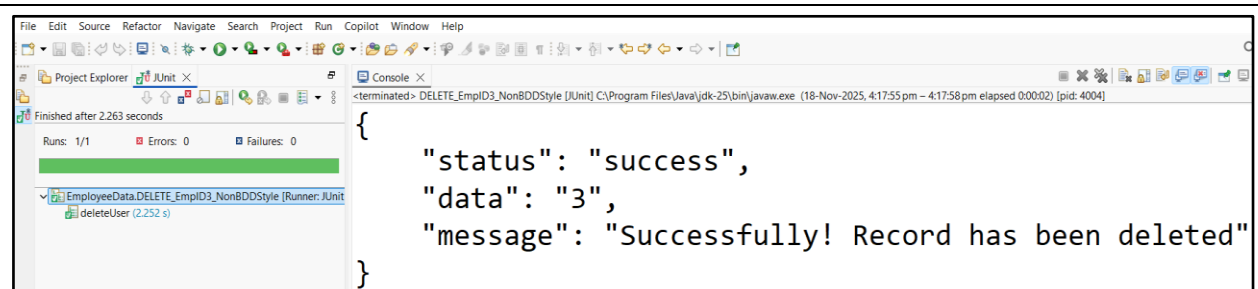
package EmployeeData;
import io.restassured.RestAssured;
import io.restassured.response.Response;
import io.restassured.response.ValidatableResponse;
import io.restassured.specification.RequestSpecification;

```

```

import org.junit.Test;
import static org.hamcrest.CoreMatchers.equalTo;
public class DELETE_EmpID3_NonBDDStyle
{
    RequestSpecification requestSpecification;
    Response response;
    ValidatableResponse validatableResponse;
    @Test
    public void deleteUser() {
        RestAssured.baseURI = "https://dummy.restapiexample.com/api";
        // Create a request specification
        requestSpecification = RestAssured.given();
        // Calling DELETE method
        response = requestSpecification.delete("/v1/delete/3");
        // Let's print response body.
        String resString = response.prettyPrint();
        /*To perform validation on response, we need to get ValidatableResponse type of
        response */
        validatableResponse = response.then();
        // Get status code
        validatableResponse.statusCode(200);
        // It will check if status line is as expected
        validatableResponse.statusLine("HTTP/1.1 200 OK");
        // Check response - message attribute
        validatableResponse.body("message", equalTo("Successfully! Record has been
        deleted"));
    } }

```



5) PATCH Request using Rest Assured:

The HTTP PATCH request method applies partial modifications to a resource. PUT is a method of modifying resources where the client sends data that updates the entire resource. PATCH is a method of modifying resources where the client sends partial data that is to be updated without modifying the entire data.

Below is an example of a PATCH Request in Postman – Update User Name:

API Endpoint: To update only the name of user with ID 2, use:

`https://reqres.in/api/users/2`

Select HTTP Method: Choose: PATCH

Add Headers: Key: Content-Type Value: application/json

Add Request Body: Go to: Body → raw → JSON: Enter:

```
{  
  "name": "William"  
}
```

This means you are partially updating only one field.

Send the Request: Click: SEND

Expected Response (Success):

```
{  
  "name": "William",  
  "updatedAt": "2025-11-07T10:45:00.000Z"  
}
```

✓ **Status Code:** 200 OK

Below are the steps to test a PATCH Request using Rest Assured implemented in BDD Format:

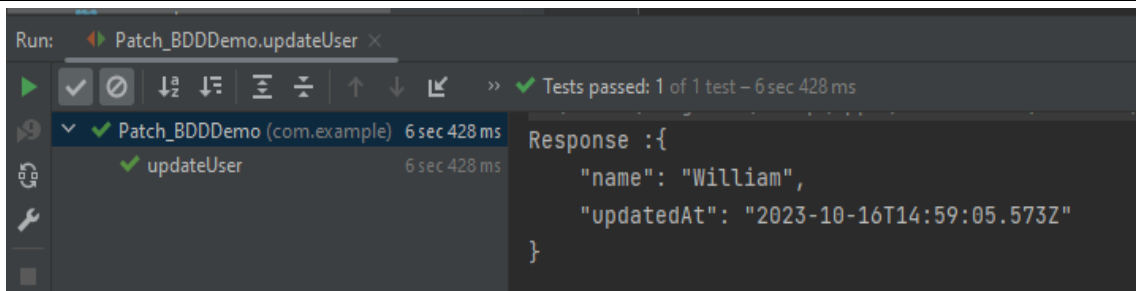
```
package EmployeeData;  
import io.restassured.http.ContentType;  
import io.restassured.response.Response;  
import io.restassured.response.ValidatableResponse;  
import io.restassured.specification.RequestSpecification;  
import org.junit.Test;  
import static io.restassured.RestAssured.given;  
import static org.hamcrest.CoreMatchers.equalTo;  
public class PATCH_Update_UserNameID2_BDDStyle {  
    RequestSpecification requestSpecification;  
}
```

```

Response response;
ValidatableResponse validatableResponse;

@Test
public void updateUser() {
    String jsonString = "{\"name\": \"William\"}";
    ValidatableResponse validatableResponse = given()
        .baseUrl("https://reqres.in")
        .basePath("/api/users/2")
        .contentType(ContentType.JSON)
        .body(jsonString)
        .when()
        .patch()
        .then()
        .statusCode(200)
        .body("name", equalTo("William"));
    System.out.println("Response: " + validatableResponse.extract().asPrettyString());
} }

```



Below is the example for the test to PATCH method. (Non BDD):

```

package EmployeeData;

import io.restassured.RestAssured;
import io.restassured.http.ContentType;
import io.restassured.response.Response;
import io.restassured.response.ValidatableResponse;
import io.restassured.specification.RequestSpecification;
import org.junit.Test;
import static org.hamcrest.CoreMatchers.equalTo;

public class PATCH_Update_UserNameID2_NonBDDStyle
{

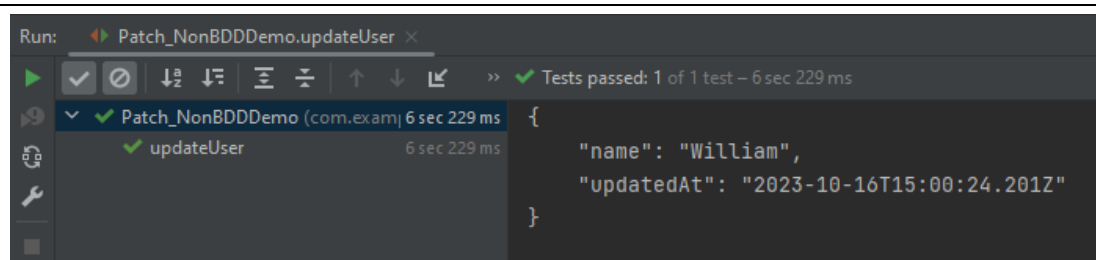
```

```

RequestSpecification requestSpecification;
Response response;
ValidatableResponse validatableResponse;

@Test
public void updateUser() {
    String jsonString = "{\"name\": \"William\"}";
    RestAssured.baseURI = "https://reqres.in/api/users/2";
    // Create a request specification
    requestSpecification = RestAssured.given();
    // Setting content type to specify format in which request payload will be
    sent.
    requestSpecification.contentType(ContentType.JSON);
    // Adding body as string
    requestSpecification.body(jsonString);
    // Calling PATCH method
    response = requestSpecification.patch();
    // Let's print response body.
    String responseString = response.prettyPrint();
    /* To perform validation on response, we need to get
    ValidatableResponse type of response */
    validatableResponse = response.then();
    // Get status code
    validatableResponse.statusCode(200);
    // It will check if status line is as expected
    validatableResponse.statusLine("HTTP/1.1 200 OK");
    // Check response - name attribute
    validatableResponse.body("name", equalTo("William"));
} }

```



<https://qaautomation.expert/2023/10/12/rest-assured-tutorials/>

<https://qaautomation.expert/2023/10/12/rest-assured-tutorials/>

Course Content:

Installation and Basic Setup

Introduction to API Automation

Setting Up the Project Structure

Dependencies We Need

Overview of Required Dependencies

Installation and Configuration

How to Automate HTTPS Requests

Understanding HTTPS

Handling HTTP/HTTPS Requests

Creating Payload for POST/PUT Requests in Multiple Ways

HashMap
JSONObject
POJO (Plain Old Java Object)
External JSON
Assertions
Importance of Assertions in API Testing
Common Assertion Techniques
How to Deal with Cookies
Understanding Cookies
Managing Cookies in API Requests
How to Deal with Headers
Header Types and Functions
Customizing Headers in Requests
Path and Query Parameters
Understanding Path Parameters
Working with Query Parameters
Validation Response
Validating Status Codes
Response Time Validation
Parsing JSON Response
Introduction to JSON Parsing
Extracting Data from JSON Responses
Validating JSON Schema
Overview of JSON Schema Validation
Implementation in API Testing
Types of Authorization
Basic Authorization
Digest Authorization
Preemptive Authorization
Bearer Token Authorization
OAuth 1.0 and 2.0
etc
Framework
Building an API Automation Framework

Best Practices and Structure

Reporting

Overview of API Test Reporting

Popular Reporting Tools and Libraries

Q&A and Project Work

Live Examples and Scenarios

Practical Project Work for Hands-on Experience

FAKER Library

Introduction to FAKER Library

Generating Random Data for Testing

API Chaining

Basics of API Chaining

Implementing API Chaining in Automation